

Indian Institute of Technology Dharwad
Department of EE/CSE/ME
Neural Networks and Deep Learning Lab

LAB-5

Feb 10, 2026

Lab Assignment: Implementing a Convolutional Neural Network (CNN)

Objective:

The objective of this lab assignment is to implement and train a Convolutional Neural Network (CNN) for image classification. Students will gain hands-on experience in building a CNN, optimizing its performance, and analyzing its results.

Instructions:

1. Use Python with TensorFlow/Keras or PyTorch.
 2. Submit a detailed report that includes:
 - Code implementation.
 - Model architecture and parameter details.
 - Training accuracy and loss plots.
 - Test accuracy and classification metrics.
 - Observations and conclusions.
 3. Ensure the code is well-commented and structured.
-

Tasks:

Task 1: Dataset Preparation

1. Use MNIST digits dataset:
 - **MNIST** (Handwritten digit classification, 28x28 grayscale images)
 2. Perform necessary preprocessing:
 - Normalize pixel values.
 - Convert labels to categorical format (if using Cross-Entropy Loss).
 - Split data into training and test sets.
-

Task 2: Implement a CNN Model

1. Build a CNN with the following layers:
 - Convolutional layers with ReLU activation.
 - Max pooling layers for downsampling.
 - Fully connected (Dense) layers.
 - Output layer with softmax activation (for multi-class classification).
2. Define the model using either TensorFlow/Keras or PyTorch.

3. Print the model summary.
-

Task 3: Train the CNN

1. Compile the model with:
 - **Loss Function:** Cross-Entropy Loss.
 - **Optimizer:** Adam or SGD.
 - **Evaluation Metric:** Accuracy.
 2. Train the model for at least 10-20 epochs.
 3. Save the model weights after training.
-

Task 4: Evaluate and Visualize Results

1. Plot training and validation accuracy/loss over epochs.
 2. Evaluate test accuracy and generate a classification report.
 3. Visualize a few misclassified images with predictions.
 4. Create a confusion matrix for deeper insight into model performance.
-

Task 5: Experimentation and Analysis

1. Compare different architectures by modifying:
 - The number of convolutional layers.
 - The number of filters per layer.
 - Different activation functions (ReLU, Leaky ReLU, etc.).
 2. Train models with and without dropout layers to analyze overfitting.
 3. Execute a hyperparameter search (Grid or Random) to optimize the batch size, learning rate and other parameters. **This process may be time consuming as it requires multiple full training cycles and make use of gpu in GoogleColab.** Report the best configuration found.
-

Additional Observations to be Demonstrated:

1. Use suitable data to demonstrate the same (MNIST digit data can be used).
2. Demonstrate the translation invariant property of the pooling layer by choosing two images, where one is translated, and show that the output of the pooling layer is nearly identical for both images.
3. Demonstrate how backpropagation works with the max pooling layer.

Note: Use only two classes and a small network. Write your own code to demonstrate the same.

Programming Task:

- Implement a CNN to demonstrate the above principles.

- Provide objectives, hypothesis, and experimental description.
 - Write a program using Python to achieve the desired results.
-

Inferences and Conclusion:

- State all key observations and insights from the experiments.
 - Compare performance across different models and configurations.
-

Submission Guidelines:

1. Submit the complete source code.
2. Provide a well-structured PDF report with:
 - Model architecture details.
 - Training performance (plots and metrics).
 - Key observations and conclusions.
3. Ensure all visualizations are clearly labeled.